

Self-Assessment Report for Neulaser Reference Implementation

June 30, 2026

1 Basic Algorithm Information

Item	Description
Algorithm category	Cryptographic hash algorithm
Algorithm name	Neulaser
Subdivided function	Fixed-output hash function
Implementation version	Reference Implementation Version
Parameter sets	Neulaser-512, Neulaser-768, Neulaser-1024
Digest sizes	512, 768, and 1024 bits
Implementation language	ISO C99

The assessed implementation is the portable reference implementation. It does not use assembly, SIMD intrinsics, multi-threading, or external cryptographic libraries.

2 Assessment Environment Information

Item	Configuration
Processor	Intel Core i7-1165G7
Nominal clock frequency	2.80 GHz
Cores/threads used	1 core used for testing; processor has 4 cores / 8 threads
Memory	16478896 KiB physical memory
Operating system	Windows NT 10.0.26200, 64-bit
Runtime environment	MSYS2 runtime 3.6.9
Instruction set support	x86-64 with SSE2, AVX, AVX2, AES-NI, SHA-NI
Compiler	GCC 15.2.0, target x86_64-w64-mingw32
Build tool	mingw32-make / GNU Make 4.4.1
Compile flags	gcc -std=c99 -Wpedantic -Wall -Wextra -O2
External cryptographic libraries	None
Test time	2026-06-30, Asia/Shanghai time zone

The x86 self-assessment guideline recommends Linux with kernel version 4.19 or later. The measurements recorded here were obtained in the Windows/MSYS2 environment above; the provided source and scripts allow the same test to be rerun on Linux x86-64.

3 Functional Testing

Known-answer-test data are provided in `Self_Assessment_Code_and_Data/Test_Vector/`.

Test-vector family	Files
KAT_2_12	KAT_2_12_Neulaser-512.txt, KAT_2_12_Neulaser-768.txt, KAT_2_12_Neulaser-1024.txt
KAT_2_23	KAT_2_23_Neulaser-512.txt, KAT_2_23_Neulaser-768.txt, KAT_2_23_Neulaser-1024.txt
KAT_2_33	KAT_2_33_Neulaser-512.txt, KAT_2_33_Neulaser-768.txt, KAT_2_33_Neulaser-1024.txt
KAT_Loop	KAT_Loop_Neulaser-512.txt, KAT_Loop_Neulaser-768.txt, KAT_Loop_Neulaser-1024.txt

The KAT generation code is included in each instance directory under `Algorithm_Source_Code/`.

4 Performance Test Method

The performance test follows the S1–S8 input sizes recommended by the x86 self-assessment guideline for hash algorithms.

Level	Message length
S1	32 bytes
S2	128 bytes
S3	512 bytes
S4	1024 bytes
S5	4096 bytes
S6	8192 bytes
S7	16384 bytes
S8	65536 bytes

For each message length and each Neulaser variant, the benchmark performs 100 measurements. Each measurement repeats the `CryptHash` call enough times to process 65536 input bytes, so each table entry is averaged over 6553600 input bytes. The input buffer is generated deterministically by the fixed xorshift rule implemented in `benchmark_hash.c`. The benchmark pins the process to one CPU core and reports both wall-clock throughput and x86 time-stamp-counter cycle counts.

5 Performance Results

Level	Bytes	512 MB/s	512 cyc/B	768 MB/s	768 cyc/B	1024 MB/s	1024 cyc/B
S1	32	6.35	421.1449	4.85	550.8215	3.90	686.9858
S2	128	13.85	193.0724	19.44	137.4982	15.77	169.4992
S3	512	23.36	114.4534	21.64	123.5642	23.42	114.1643
S4	1024	25.84	103.4656	25.40	105.2658	24.40	109.5463
S5	4096	27.37	97.6781	25.58	104.5280	25.92	103.1369
S6	8192	27.85	95.9961	26.72	100.0441	26.23	101.9026
S7	16384	28.20	94.7930	26.75	99.9404	26.42	101.1736
S8	65536	28.25	94.6488	26.74	99.9613	26.74	99.9664

The CSV version of this table is provided in the self-assessment code and data directory:

`Self_Assessment_Code_and_Data/performance_results_reference.csv`

6 Resource and Dependency Summary

The reference implementation is pure C99 and uses no external cryptographic libraries. The dominant resource is the in-memory PRNG state, whose sizes are 1536, 2048, and 2560 bits for the three parameter sets. The implementation uses fixed-size stack buffers for message-block processing and output generation; no heap allocation is used by the `CryptHash` implementation.